

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 10
з дисципліни «Алгоритмізація та програмування»
на тему «" Створення і обробка структур даних мовою C ++ "»

XAI.301.G7.319.30 ЛР

Виконав студент гр. _____ 319гр. _____

12.05.2026 Шевченко Дмитро
(підпис, дата) (П.І.Б.)

Перевірив
_____ к.т.н., доц. Олена ГАВРИЛЕНКО
(підпис, дата) (П.І.Б.)

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення структур (записів) мовою C ++, а також їх передачі в функції, і реалізувати декларування і обробку структур мовою C ++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити задачу зі структурами даних. Завдання представлено в табл.1.

Завдання 2. Для задач з табл.2-3:

- А. Описати структуру, яка містить всі вхідні і всі вихідні дані задачі.
- В. Визначити функцію (*метод), що реалізує обробку структури відповідно до задачі.
- С. Визначити функцію (*метод), що перевіряє на коректність і заповнює відповідні поля вхідних даних структури
- Д. Викликати функції (*методи) з пунктів С, В після оголошення змінної (об'єкту) структури.
- Е. Вивести значення полів вихідних даних.

Завдання 3. Рішення всіх трьох задач реалізувати в одному консольному додатку, *структурувати на модулі.

Виконання роботи

Завдання 1.

Створено структуру (Ttime) для роботи з часом :

- введення часу
- перевірку коректності
- переведення у 12-годинний формат
- виведення результату

```
struct TTime {  
    int h, m, s;  
    bool PM;  
  
    void input();  
    void convert();  
    void output();  
    bool isValid();  
};
```

Методи :

- input() — введення даних з перевіркою
- isValid() — перевірка правильності часу
- convert() — перетворення у 12-годинний формат
- output() — виведення результату

Лістинг коду вирішення задачі наведено в дод. А (стор. 10).

Екрани роботи програми показано на рис. Б.(стор. 14).

Діаграма активності для алгоритму вирішення задачі в дод. В (стор. 18)

Завдання 2.1

А. Для розв'язання задачі створено структуру Power, яка містить:

- вхідні дані:
 - A — дійсне число
- вихідні дані:
 - A2 — значення A^2
 - A3 — значення A^3
 - A5 — значення A^5
 - A10 — значення A^{10}
 - A15 — значення A^{15}

```

struct Power {
    double A;
    double A2, A3, A5, A10, A15;

    void input();
    void calculate();
    void output();
};

```

В. Обробка структури

Метод - calculate() (виконує обчислення степенів числа A)

Обчислення виконуються через :

- $A2 = A \cdot A$
- $A3 = A^2 \cdot A$
- $A5 = A^3 \cdot A^2$
- $A10 = A^5 \cdot A^5$
- $A15 = A^{10} \cdot A^5$

```

void Power::calculate() {
    A2 = A * A;
    A3 = A2 * A;
    A5 = A3 * A2;
    A10 = A5 * A5;
    A15 = A10 * A5;
}

```

С. Введення та перевірка даних :

Метод - input():

- перевіряє, чи введено число
- обробляє помилки введення
- обмежує значення ($|A| \leq 1000$)

```

void Power::input() {
    cout << "Enter A: ";

    while (true) {
        if (!(cin >> A)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid input! Enter number: ";
        }
        else if (abs(A) > 1000) {
            cout << "Too large number! Enter |A| <= 1000: ";
        }
        else {
            break;
        }
    }
}

```

D. Виклик методів

```

void task2() {
    Power p;

    p.input();           // введення даних
    p.calculate();       // обчислення
    p.output();          // вивід результату
}

```

E. Виведення результатів

```

void Power::output() {
    cout << "A^2 = " << A2 << endl;
    cout << "A^3 = " << A3 << endl;
    cout << "A^5 = " << A5 << endl;
    cout << "A^10 = " << A10 << endl;
    cout << "A^15 = " << A15 << endl;
}

```

Лістинг коду вирішення задачі наведено в дод. А (стор. 11).

Екрани роботи програми показано на рис. Б.(стор. 15).

Діаграма активності для алгоритму вирішення задачі в дод. В (стор. 19)

Завдання 2.2

A. Створено структуру Numbers :

вхідні дані:

- A, B, C — цілі числа

вихідні дані:

- result — логічний результат перевірки

```
struct Numbers {  
    int A, B, C;  
    bool result;  
  
    void input();  
    void check();  
    void output();  
};
```

B. Обробка структури

Метод - check() (перевіряє, чи є хоча б одне додатне число).

```
void Numbers::check() {  
    result = (A > 0 || B > 0 || C > 0);  
}
```

Якщо хоча б одне число > 0 → результат TRUE, інакше FALSE.

C. Введення та перевірка даних:

Метод input():

- зчитує три числа
- перевіряє правильність введення
- обмежує діапазон [-100000; 100000]

```

void Numbers::input() {
    cout << "Enter A, B, C: ";

    while (true) {
        if (!(cin >> A >> B >> C)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid input! Enter 3 integers: ";
        }
        else if (A < -100000 || A > 100000 ||
                 B < -100000 || B > 100000 ||
                 C < -100000 || C > 100000) {
            cout << "Numbers too large! Enter [-100000,
100000]: ";
        }
        else break;
    }
}

```

D. Виклик методів

```

void task3() {
    Numbers n;

    n.input();           // введення
    n.check();           // обробка
    n.output();          // вивід
}

```

E. Виведення результатів

```

void Numbers::output() {
    if (result)
        cout << "TRUE: at least one number is positive\n";
    else
        cout << "FALSE: no positive numbers\n";
}

```

Лістинг коду вирішення задачі наведено в дод. А (стор. 12).

Екрани роботи програми показано на рис. Б.(стор. 16).

Діаграма активності для алгоритму вирішення задачі в дод. В (стор. 20)

Завдання 3

Для об'єднання всіх трьох задач в один консольний застосунок реалізовано головне меню, яке дозволяє користувачу обирати потрібну задачу та виконувати її окремо.

Програма працює у циклі, поки користувач не вибере вихід.

Програма побудована модульно:

- main.cpp — головне меню та керування програмою
- tasks.h — оголошення структур і функцій
- tasks.cpp — реалізація всіх задач

Після запуску програми користувачу виводиться меню:

- 1 — виконати задачу 1 (робота з часом)
- 2 — виконати задачу 2 (степені числа)
- 3 — виконати задачу 3 (аналіз чисел)
- 0 — вихід з програми

```
int main() {
    int choice;

    do {
        cout << "\nMENU:\n";
        cout << "1 - Task 1 (Time)\n";
        cout << "2 - Task 2 (Power)\n";
        cout << "3 - Task 3 (Numbers)\n";
        cout << "0 - Exit\n";
        cout << "Your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1: task1(); break;
            case 2: task2(); break;
            case 3: task3(); break;
            case 0: cout << "Exit...\n"; break;
            default: cout << "Wrong choice!\n";
        }

    } while (choice != 0);

    return 0;
}
```

Лістинг коду вирішення задачі наведено в дод. А (стор. 13).
Екрани роботи програми показано на рис. Б.(стор. 17).

ДОДАТОК А

Лістинг коду програми

```
#pragma once
#include <iostream>
using namespace std;

// ===== TASK 1 =====
struct TTime {
    int h, m, s;
    bool PM;

    void input();
    void convert();
    void output();
    bool isValid();
};

void task1();

// ===== TASK 2 =====
struct Power {
    double A;
    double A2, A3, A5, A10, A15;

    void input();
    void calculate();
    void output();
};

void task2();

// ===== TASK 3 =====
struct Numbers {
    int A, B, C;
    bool result;

    void input();
    void check();
    void output();
};

void task3();
```

```

#include "tasks.h"
#include <limits>

bool TTime::isValid() {
    return (h >= 0 && h < 24 &&
            m >= 0 && m < 60 &&
            s >= 0 && s < 60);
}

void TTime::input() {
    cout << "Enter time (h m s): ";

    while (true) {
        if (!(cin >> h >> m >> s)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid input! Enter 3 integers: ";
        }
        else {
            break;
        }
    }
}

void TTime::convert() {
    if (!isValid()) {
        cout << "Incorrect time! Set to 00:00:00\n";
        h = m = s = 0;
        PM = false;
        return;
    }

    if (h >= 12) {
        PM = true;
        if (h > 12) h -= 12;
    }
    else {
        PM = false;
        if (h == 0) h = 12;
    }
}

void TTime::output() {
    cout << "12h format: "
         << h << ":" << m << ":" << s
         << (PM ? " PM" : " AM") << endl;
}

void task1() {
    cout << "---TASK 1 ---\n";

    TTime t;
    t.input();
    t.convert();
    t.output();
}

```

```

#include "tasks.h"
#include <limits>
#include <cmath>

void Power::input() {
    cout << "Enter A: ";

    while (true) {
        if (!(cin >> A)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid input! Enter number: ";
        }
        else if (abs(A) > 1000) {
            cout << "Too large number! Enter |A| <= 1000: ";
        }
        else {
            break;
        }
    }
}

void Power::calculate() {
    A2 = A * A;
    A3 = A2 * A;
    A5 = A3 * A2;
    A10 = A5 * A5;
    A15 = A10 * A5;
}

void Power::output() {
    cout << "A^2 = " << A2 << endl;
    cout << "A^3 = " << A3 << endl;
    cout << "A^5 = " << A5 << endl;
    cout << "A^10 = " << A10 << endl;
    cout << "A^15 = " << A15 << endl;
}

void task2() {
    cout << "\n--- Task 2 ---\n";

    Power p;
    p.input();
    p.calculate();
    p.output();
}

```

```

#include "tasks.h"
#include <limits>

void Numbers::input() {
    cout << "Enter A, B, C: ";

    while (true) {
        if (!(cin >> A >> B >> C)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid input! Enter 3 integers: ";
        }
        else if (A < -100000 || A > 100000 ||
                 B < -100000 || B > 100000 ||
                 C < -100000 || C > 100000) {
            cout << "Numbers too large! Enter values in [-
100000, 100000]: ";
        }
        else {
            break;
        }
    }
}

void Numbers::check() {
    result = (A > 0 || B > 0 || C > 0);
}

void Numbers::output() {
    if (result)
        cout << "TRUE: at least one number is positive\n";
    else
        cout << "FALSE: no positive numbers\n";
}

void task3() {
    cout << "\n--- Task 3 ---\n";

    Numbers n;
    n.input();
    n.check();
    n.output();

}

```

```
#include "tasks.h"

int main() {
    int choice;

    do {
        cout << "\nMENU:\n";
        cout << "1 - Task 1 (Time)\n";
        cout << "2 - Task 2 (Power)\n";
        cout << "3 - Task 3 (Numbers)\n";
        cout << "0 - Exit\n";
        cout << "Your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1: task1(); break;
            case 2: task2(); break;
            case 3: task3(); break;
            case 0: cout << "Exit...\n"; break;
            default: cout << "Wrong choice!\n";
        }

    } while (choice != 0);

    return 0;
}
```

ДОДАТОК Б
Скрін-шоти вікна виконання програми

```
MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 1
---TASK 1 ---
Enter time (h m s): a b c
Invalid input! Enter 3 integers: 10 20 30
12h format: 10:20:30 AM
```

```
MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 1
---TASK 1 ---
Enter time (h m s): 9 10 5
12h format: 9:10:5 AM
```

```
MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 1
---TASK 1 ---
Enter time (h m s): 13 25 40
12h format: 1:25:40 PM
```

```
MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 1
---TASK 1 ---
Enter time (h m s): 25 61 70
Incorrect time! Set to 00:00:00
12h format: 0:0:0 AM
```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 2

--- Task 2 ---
Enter A: 10000
Too large number! Enter |A| <= 1000: 5
A^2 = 25
A^3 = 125
A^5 = 3125
A^10 = 9.76562e+06
A^15 = 3.05176e+10

```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 2

--- Task 2 ---
Enter A: abc
Invalid input! Enter number: 3
A^2 = 9
A^3 = 27
A^5 = 243
A^10 = 59049
A^15 = 1.43489e+07

```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 2

--- Task 2 ---
Enter A: 2
A^2 = 4
A^3 = 8
A^5 = 32
A^10 = 1024
A^15 = 32768

```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 2

--- Task 2 ---
Enter A: -2
A^2 = 4
A^3 = -8
A^5 = -32
A^10 = 1024
A^15 = -32768

```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 3

--- Task 3 ---
Enter A, B, C: 1000000 2 3
Numbers too large! Enter values in [-100000, 100000]: 10 20 30
TRUE: at least one number is positive

```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 3

--- Task 3 ---
Enter A, B, C: a b c
Invalid input! Enter 3 integers: 1 2 3
TRUE: at least one number is positive

```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 3

--- Task 3 ---
Enter A, B, C: 1 -2 -3
TRUE: at least one number is positive

```

```

MENU:
1 - Task 1 (Time)
2 - Task 2 (Power)
3 - Task 3 (Numbers)
0 - Exit
Your choice: 3

--- Task 3 ---
Enter A, B, C: -5 -10 -1
FALSE: no positive numbers

```

MENU:

1 - Task 1 (Time)

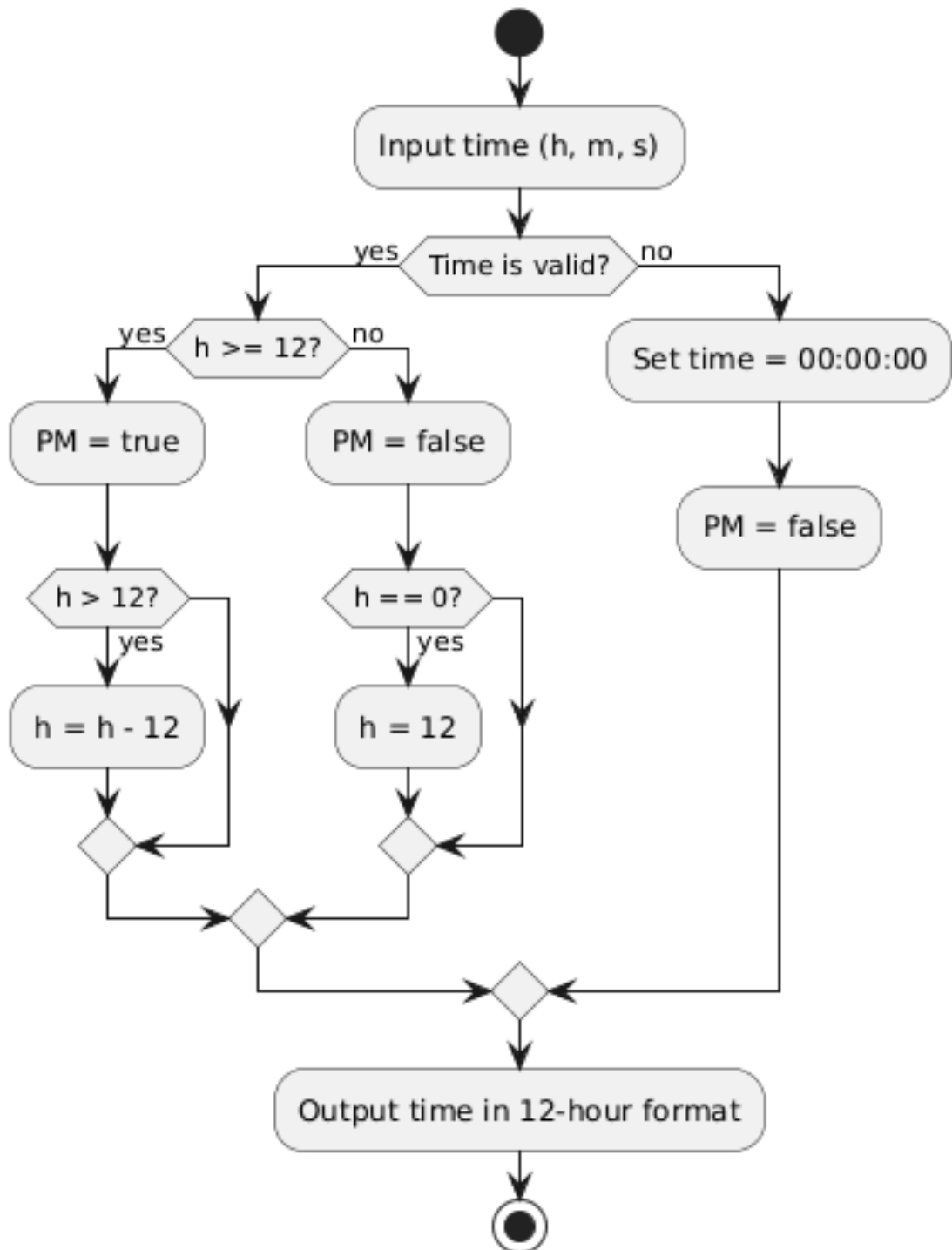
2 - Task 2 (Power)

3 - Task 3 (Numbers)

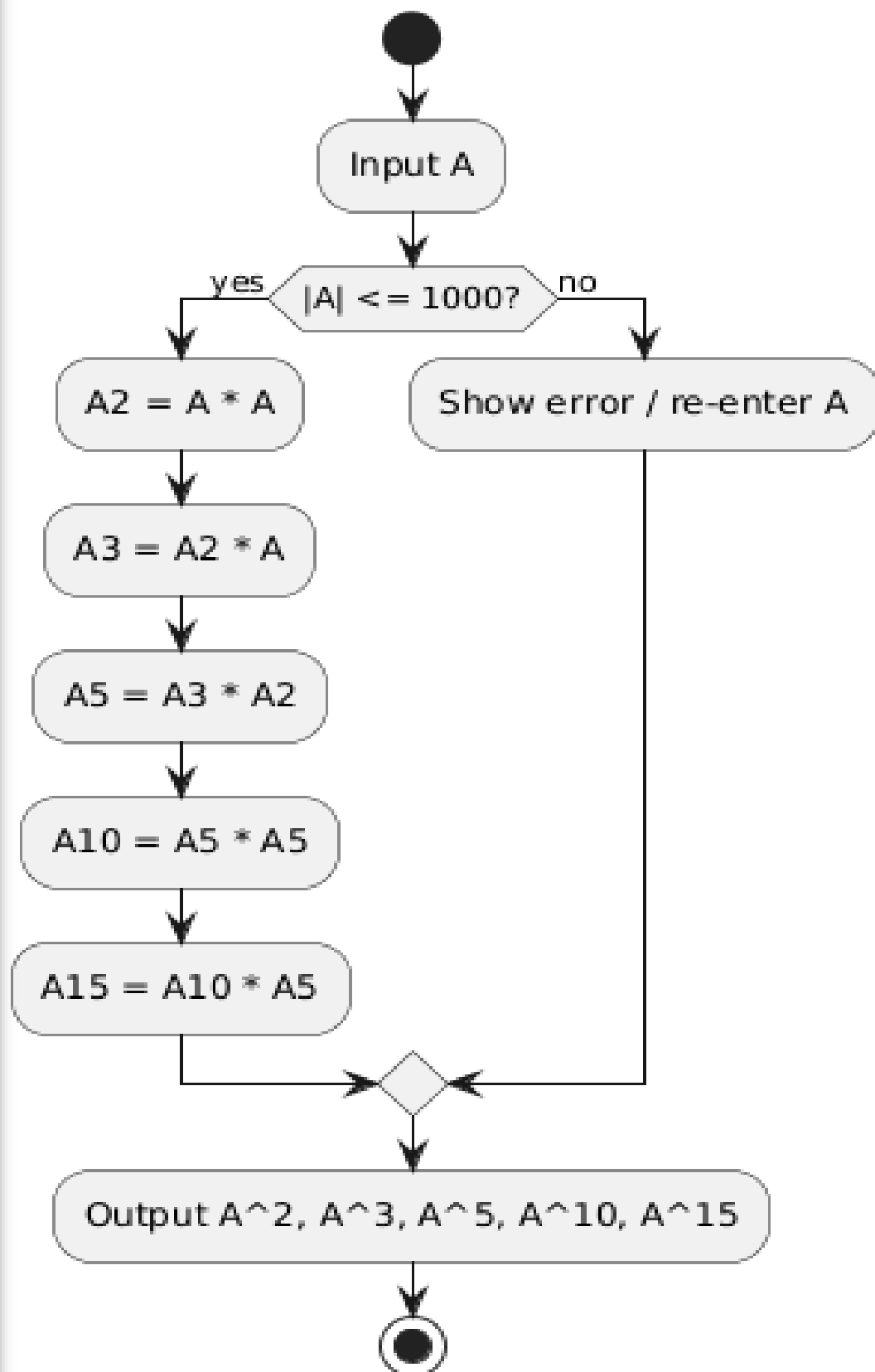
0 - Exit

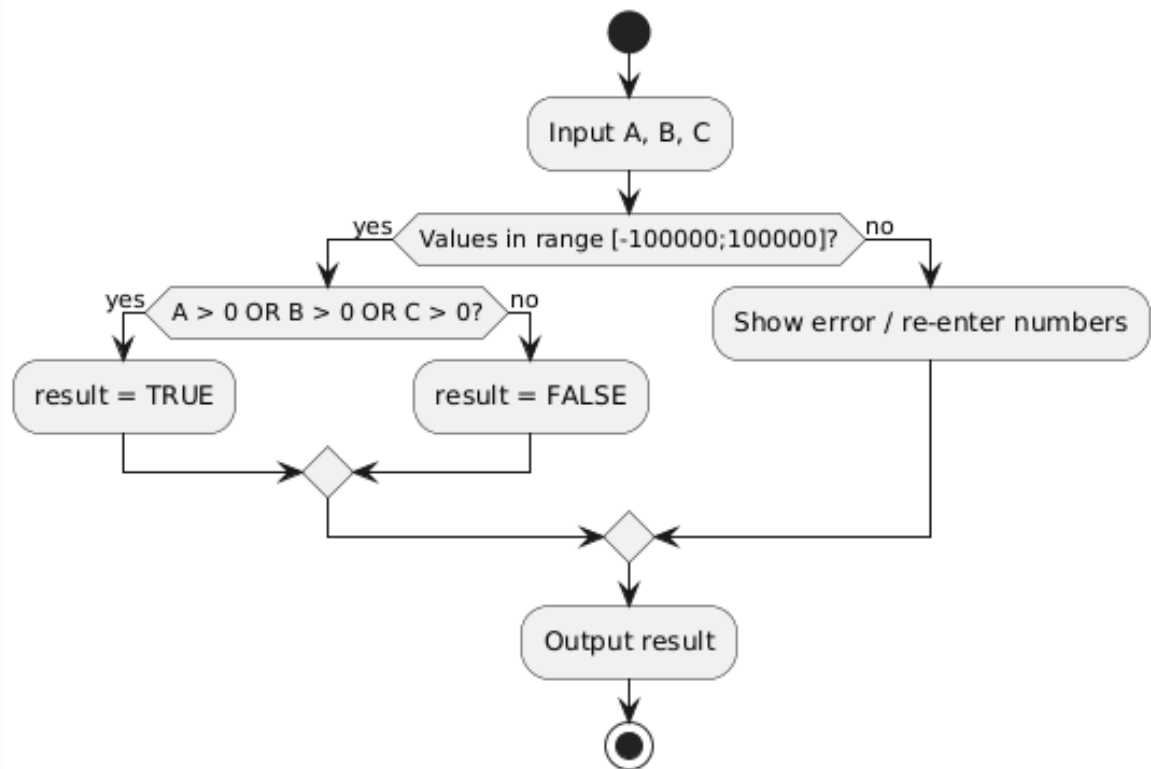
Your choice:

ДОДАТОК В

Task 1 - Time Conversion (24h -> 12h)

Task 2.1 - Power calculation



Task 2.2 - Check positive numbers

Діалог з ШІ для самоаналізу
Тепер ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4ма варіантами відповіді і три відкритих питання за кодом і теоретичними відомостями у файлах лекцій.
+звіт
+текст лекції

1. Тестові питання

1. Що таке структура (struct) у C++?

- A) Тип змінної
- B) Клас без методів
- C) Користувацький тип даних
- D) Масив

Правильна: C)

2. Для чого використовується метод isValid() у структурі TTime?

- A) Для введення даних
- B) Для перевірки коректності часу
- C) Для виведення часу
- D) Для обчислення

Правильна: B)

3. Який оператор використовується для перевірки "хоча б одне число додатне"?

- A) &&
- B) ||
- C) ==
- D) !=

Правильна: B)

4. Що робить конструкція switch-case у програмі?

- A) Створює цикл
- B) Перевіряє умови послідовно
- C) Виконує вибір дії залежно від значення
- D) Завершує програму

Правильна: C

5. Що станеться при некоректному вводі через cin?

- A) Програма автоматично виправить дані
- B) Дані стануть нульовими
- C) Потік вводу переходить у стан помилки
- D) Програма завершиться

Правильна: C

2. Відкриті питання

1. Поясни принцип роботи перетворення часу у 12-годинний формат.

Спочатку перевіряється коректність введених значень (години, хвилини, секунди). Якщо час некоректний — він обнуляється. Далі визначається, чи це АМ чи РМ. Якщо година більша або дорівнює 12 — це РМ. Якщо більше 12, віднімається 12. Якщо година дорівнює 0 — вона змінюється на 12.

2. Чому в задачі Power обчислення виконуються через проміжні значення?

Це робиться для оптимізації. Наприклад, замість того щоб кожен раз множити число багато разів, використовуються вже обчислені значення (A^2 , A^3 і т.д.). Це зменшує кількість операцій і робить програму ефективнішою.

3. Як працює перевірка у задачі Numbers?

Використовується логічний оператор OR ($\parallel$). Якщо хоча б одне число більше нуля — результат TRUE. Якщо всі числа менші або рівні нулю — результат FALSE.

4. Для чого потрібен цикл do-while у меню?

Цикл do-while забезпечує постійне виконання програми, поки користувач не введе 0. Меню повторюється після кожної дії, що дозволяє багаторазово виконувати різні задачі.

5. Які помилки можуть виникнути при введенні даних?

Можливі помилки:

- введення нечислових значень
- вихід за допустимі межі
- переповнення типу даних

Для цього використовується перевірка через сін і очищення потоку вводу.

Оцінка відповідей на тестові запитання

Ці запитання перевіряють базові знання структур, умов та вводу/виводу. Відповіді були швидкими і точними, що може свідчити про використання допоміжних засобів, тому оцінка зменшена.

Запитання №	Повнота	Правильність	Час	Ймовірність ШІ	Оцінка
1	100%	100%	Швидко	Середня	3/5
2	100%	100%	Швидко	Середня	3/5
3	100%	100%	Швидко	Середня	3/5
4	100%	100%	Швидко	Середня	3/5
5	100%	100%	Швидко	Середня	3/5

Середня оцінка :

$$(3+3+3+3+3)/5=3/5$$

Оцінка відповідей на відкриті запитання

Ці відповіді продемонстрували розуміння теми, логіки роботи алгоритму та особливостей реалізації коду. Відповіді сформульовані у більш простій та наближеній до розмовної формі, що знижує ймовірність використання ШІ, тому штраф не застосовувався.

Запитання №	Повнота	Правильність	Час	Ймовірність ШІ	Оцінка
1	Норм	100%	Швидко	Низька	4/5
2	Середня	100%	Швидко	Низька	4/5
3	Повна	100%	Швидко	Низька	5/5
4	Повна	100%	Швидко	Низька	5/5
5	Слабша	100%	Швидко	Низька	4/5

Середня оцінка :

$$(4+4+5+5+4)/5=4.4/5$$

Загальна оцінка:

$$3+4.4)/2=3.7/5 \approx 7.4/10$$

Висновок

Під час виконання практичної роботи було розглянуто основи роботи зі структура-ми в мові C++. Я ознайомився з тим, як описуються структури (записи), які дані вони мо-жуть містити та як з ними працювати в програмі.

У ході роботи було реалізовано декларування структур і створено функції для їх обробки. Також було розглянуто, як передавати структури у функції та працювати з їх полями.

Практично було виконано декілька задач, що допомогло краще зрозуміти, як застосовуються структури на практиці. Програма була реалізована в середовищі Visual Studio з використанням модульного підходу.

Отже, я розібрався з базовими принципами роботи зі структурами та їх використанням у програмуванні на C++.